THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Chosen Software: VLC Media Player | License: GPL v2

Student Name	R Harsha Reddy
Registration Number	24MEI10065
Slot	B22
Date of Submission	31 March 2026
Course	*Open Source Software* | *OSS NGMC*

Part A — Origin and Philosophy

A1. The Problem This Software Was Created to Solve

VLC media player launched in 2001 and made digital video playback easier for all users. Users faced major difficulties when they tried to watch videos or stream content through their computers before VLC media player became accessible. The media players of the late 1990s operated with strict limitations because they used specific codecs that existed within their closed systems. The media player refused to open the downloaded video file because it could not recognize the video file's codec which the user was trying to access. The network video streaming problem required a solution because there were no available free tools for users to stream video through the network. The late 1990s required universities to establish a standard for their systems while they required their systems to support multiple formats.

The École Centrale Paris students required a video streaming solution which lead to the creation of VLC media player. The software options available at that time failed to meet student requirements because they either charged high prices or operated with strict controls or lacked sufficient open-source features for necessary modifications.

VLC established its identity through its operational capabilities and its core value system. The VideoLAN Project started as an open-source project according to its creators. The world received access to the program because its developers decided to keep it free instead of restricting its usage through proprietary rights. The software project allowed international developers to increase codec support and create a more stable product. The VLC player developed into the ultimate solution for users who wanted to play all types of media content which had become fragmented across multiple formats and ecosystems.

VLC solved the main problem which prevented users from accessing content.

A2. The License — What It Actually Says

The Four Freedoms of Free Software

The VLC media player software is covered by the GNU General Public License, which is a commonly used free software license for software packages. It was developed by Richard Stallman, who is the founder of the Free Software Foundation. It is intended to assure users of their basic freedom in using the software. These freedoms include four main freedoms: the freedom to use the software for any purpose, the freedom to study how the software works, the freedom to modify the software to meet their needs, and the freedom to

distribute the software in both its original form and in modified form. All of these freedoms are granted by the VLC media player software, which qualifies it as free software.

Can a company take this software, modify it, and sell it without sharing their changes?

The main feature of the GNU GPL license provides software developers with copyright protection through its copyleft provision which controls their ability to distribute modified software. A company is at liberty to make use of VLC, modify it if they wish, and even sell it. The company is forbidden from generating profits through the sale of any altered software products. The company must show its customers all software changes while using the same licensing terms for all software products that it distributes. The company must provide access to the source code of all modified software products while using its same licensing terms for all software distribution.

GPL vs MIT — Differences and Your Choice

The MIT license and other more liberal licenses show a complete difference from this particular license. The GPL requires that if modifications are made to software, then it too has to be open-source. The MIT license allows programmers to use software for their projects while they can create new software and distribute it without sharing its source code. This situation exists because two distinct belief systems lead to two opposing results. The first requirement permits users to access any version of the software while the second requirement of the software demands that it should work in various environments even when those environments do not support open-source technology. This aspect holds significance for VLC software which functions based on its core principle of community-based development and transparent access.

Free as in Beer vs Free as in Freedom

The final point needs to establish that “free as in free beer” and “free as in freedom” represent two distinct meanings. The first term describes items that people can obtain without paying while the second term describes items that users can modify and share with others. VLC is both. Users have the ability to access the software at no cost, which permits them to modify and share the software with others. This is true even if it is part of other applications that are for sale. This is due to the rights that users have.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The concept of open-source software creates an attractive proposition for people who want to use it. The determination of whether every software program should become open source presents a challenging dilemma. Open-source software provides the best method for establishing open systems which enable collaborative work. The platform allows all users to access the software code which they can study for their development needs. The requirement holds relevance for critical fields such as education and security work. The

requirement exists because some software programs need to operate with their entire codebase kept secret. Software development projects require protection of their confidential data which includes both sensitive and proprietary information. The promotion of open-source software becomes harmful in situations which require this restriction. Companies that create specialized software solutions need to protect their proprietary technology through intellectual property rights. People need to understand that open-source software functions as a vital principle for software development. The current situation demonstrates that the development of both open source and closed source software holds greater significance.

Is It Ethical to Profit From Free Community Labor?

The process of generating profits through open-source software systems creates complex ethical dilemmas which require resolution. The use of Linux as an open-source foundation by companies like Red Hat Google and Meta raises a fundamental question about the ethics of earning profits from software developed by open-source communities. The reason for this exists because most companies which operate their business through open-source software Red Hat included create software development contributions which benefit the open-source community. The community receives their support through financial contributions which enables the community to sustain their existence, because they need to repay the community for the benefits it provides. The problem exists because businesses take advantage of the community resources while they fail to return any value, which creates a situation where they gain from work done by volunteers.

Standing on the Shoulders of Giants

The phrase "a man can stand on the shoulders of giants," which Sir Isaac Newton used to explain his research about Galileo Galilei, serves to explain how people create new inventions by building upon previous work. The phrase describes open-source software which enables users to access and build upon the work created by previous developers. People who use existing technologies that someone else already developed create a situation where innovation decreases according to the argument. The process of innovation generates new solutions which stem from existing inventions because open-source software generates new ideas when users access its existing content.

Part B — Linux Footprint on a Linux System

Installation Method

The VLC media player integrates into a Linux system in a relatively light and user-friendly manner, as expected from a media player designed as a desktop application and not as a service. The user can install VLC media player on Linux systems. The user can install VLC media player on Ubuntu or Debian systems through their package manager. The user needs to enter the command **sudo apt install vlc** to complete this installation. The user can install the software on RPM-based distributions through the command **dnf install vlc** or through Snap or Flatpak packages. The standard user does not need to perform manual source compilation to create the package. The media player installation process will use package manager to handle all required dependencies while following the system's established directory installation method.

Key Directories

The system has installed all VLC media player components according to standard directory locations. The media player executable `vlc` exists at the path **/usr/bin/vlc**, which permits execution from any part of the system. The media player needs its required plugins and libraries, which are kept in system architecture-dependent directories that include **/usr/lib/vlc/** and **/usr/lib64/vlc/.** The media player configuration files do not have an established central storage location, but each user maintains their own configuration directory at **~/.config/vlc/.** The media player generates only minimal log files in **/var/log/** because it operates as a non-service program that needs no extensive user interactions.

User and Group — Security Context.

VLC does not operate as a separate user or group, unlike server software. It simply runs as the user that executes it. This is a significant security practice, as VLC works with media files that are not necessarily trusted. Running it as a non-privileged user helps prevent damage if a vulnerability in VLC is exploited. In this way, the security of VLC follows the principle of least privilege, a fundamental principle of Linux security

Service Management Commands.

VLC does not operate as a service, managed by the system's service manager or init system, as it does not need to operate in the background. It can be started and stopped by the user through the command line or a GUI. However, for more experienced users, VLC can be operated in a scripted fashion or in "headless mode" for streaming, and it can then be managed as a service. In this way, a user can define a service that can be started or stopped with `systemctl --user start vlc` or `systemctl --user stop vlc`, though this is not part of a standard VLC installation

Update Model — From Patch to Your Machine

VLC's updates are managed through the package management system of the Linux operating system. When a new version of VLC or a security patch for it is released by the VideoLAN Project, the package maintainers for the operating system package up the update and distribute it through official package repositories. Users of the operating system can then update VLC through standard update tools such as `sudo apt update && sudo apt upgrade`. In this way, security patches are distributed in a way that is managed and verified, and VLC works as part of the larger system of operating system updates.

Part C — The FOSS Ecosystem

What other open-source tools does it depend on?

VLC player is dependent on a wide range of open-source libraries and system components to provide its 'play anything' feature. VLC player is based on multimedia frameworks such as FFmpeg, which is used to encode and decode different audio and video formats. VLC player also uses libraries such as libdvdcss, which is used to play DVDs with CSS encryption, and X11 or Wayland, which is used to render graphical output in Linux systems. VLC player also uses different components of the GNU toolchain, which include compilers such as GCC and autotools, which is used during the compilation of VLC player code. The different components of VLC player show that VLC player is not an independent creation but is based on the integration of different open-source technologies.

What has it enabled or inspired?

VLC player has also enabled and inspired different projects and use cases in the open-source ecosystem. VLC player is used as a platform to develop custom media player applications, IPTV applications, and different types of media player applications. VLC player is not forked often to develop different and independent applications, but its underlying framework, libVLC, is used by different developers to develop their own applications with different types of media player capabilities. VLC player is used to develop different types of mobile applications, and different types of applications related to surveillance and broadcasting also use VLC player components.

Where does it fit in the LAMP stack, or how does it relate to the web?

VLC is not directly related to the LAMP stack, since it is not a web server or database-driven system. Nevertheless, VLC is related to the web and network ecosystem, where streaming protocols are implemented, and VLC supports protocols such as HTTP, RTP, and RTSP. This enables VLC to be both a client and a streaming server. As such, VLC is related to the web ecosystem, where the VLC media player is utilized to test streaming protocols or act as a bridge between the source and the web-based system. Therefore, VLC is related to the web ecosystem but not through the backend system; VLC is related to the web ecosystem through its ability to handle multimedia data.

Is there a community

The VideoLAN project as a non-profit organization that operates internationally provides funding for the VLC project. The worldwide support of the VLC project functions as a main feature that defines its identity. The VLC project receives its backing through the VideoLAN project which exists as an international non-profit organization. The VLC project receives its backing through the VideoLAN project which exists as an international non-profit organization.

Part D — Open Source vs Proprietary (Critical Analysis)

Comparison Table: Linux Kernel vs Windows / macOS

Dimension	Open Source (VLC Media Player)	Proprietary Alternative (Windows Media Player)
Cost	Completely free to download, use, and distribute with no licensing fees.	Free with Windows OS, but tied to the operating system; not independently distributed.
Security (code audit)	Source code is publicly available; anyone can inspect, audit, and identify vulnerabilities.	Source code is closed; only the company (Microsoft) can audit and fix issues internally.
Support & reliability	Community-driven support via forums and documentation; reliable but depends on community contributions.	Official support from Microsoft; structured updates and customer support, often more predictable for enterprise use.
Freedom to modify	Full freedom to modify, redistribute, and customize under GPL license.	No access to source code; users cannot modify or redistribute the software.
Community vs corporate control	Governed by the VideoLAN Project with global contributors; decisions are community-oriented.	Fully controlled by Microsoft; development direction depends on corporate priorities.
Overall verdict	Highly flexible, transparent, and independent; ideal for users who value control, compatibility, and openness.	More integrated and polished within its ecosystem; suitable for users who prefer simplicity and official support over flexibility.

Shell Script Tasks

Script 1 — System Identity Report

Concepts: variables, echo, command substitution $(), basic output formatting.

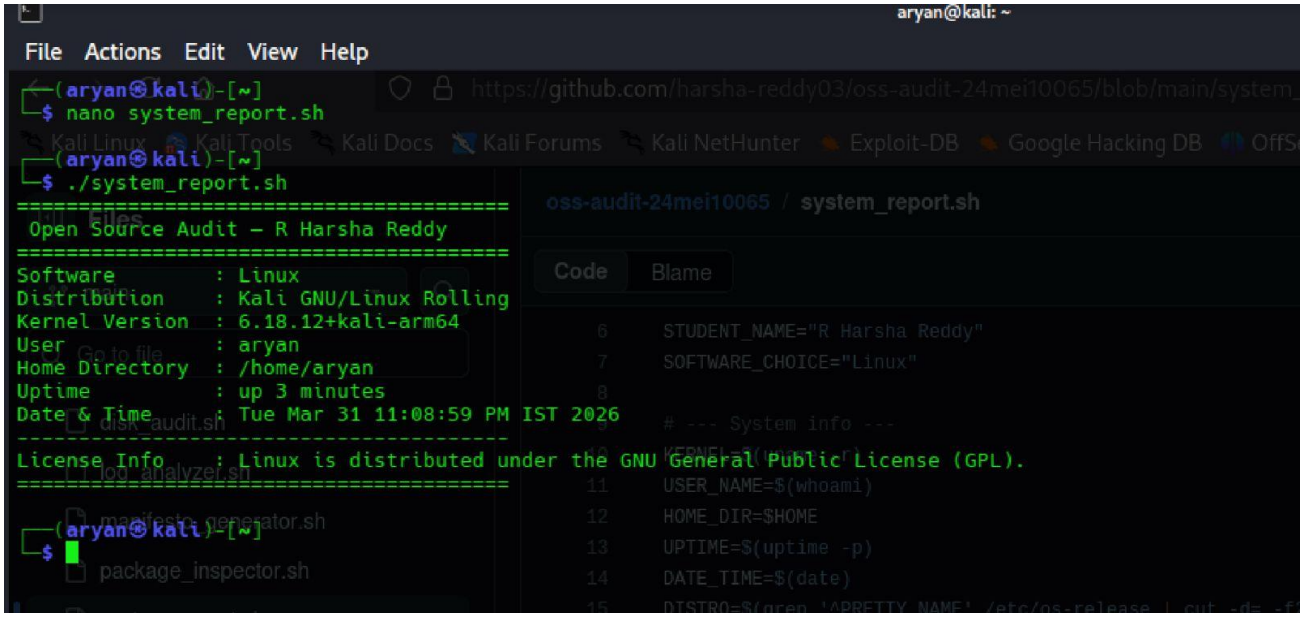
```
#!/bin/bash
# Script 1: System Identity Report
# Author: R Harsha Reddy | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="R Harsha Reddy"
SOFTWARE_CHOICE="VLC Media Player "

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE_TIME=$(date)
DISTRO=$(grep '^PRETTY_NAME' /etc/os-release | cut -d= -f2 | tr -d '"')

# --- Display ---
echo "========================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================"
echo "Software        : $SOFTWARE_CHOICE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "User            : $USER_NAME"
echo "Home Directory  : $HOME_DIR"
echo "Uptime          : $UPTIME"
echo "Date & Time     : $DATE_TIME"
echo "----------------------------------------"
echo "License Info    : Linux is distributed under the GNU General Public License
(GPL)."
echo "========================================"
```

This Bash script acts like a simple “welcome screen” for a Linux system by displaying important system details in a clear, formatted way. It collects information such as the Linux distribution name, kernel version, current user, home directory, system uptime, and current date/time. The script uses variables to store these values (e.g., kernel, user, uptime), making the code organized and reusable. It uses command substitution $( ) to execute system commands like uname, whoami, uptime, and date, and capture their outputs. The echo command is used to print all the information to the terminal. For better readability, output formatting is applied using separator lines made of = and -, which give the output a structured, welcome-screen appearance.



Script 2 — FOSS Package Inspector

Concepts: if-then-else, case statement, dpkg / rpm, pipe with grep.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: R Harsha Reddy

PACKAGE="vlc"   # Change this if needed

# Check if package is installed (works for Debian/Ubuntu systems)
if dpkg -l | grep -w $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    dpkg -l | grep -w $PACKAGE
else
    echo "$PACKAGE is NOT installed."
fi

echo "----------------------------------------"

# Case statement for package description
case $PACKAGE in
    httpd) echo "Apache: the web server that built the open internet" ;;
    mysql) echo "MySQL: open source at the heart of millions of apps" ;;
    vlc) echo "VLC: a powerful open-source media player for all formats" ;;
    firefox) echo "Firefox: privacy-focused open-source web browser" ;;
    gimp) echo "GIMP: free and open-source image editing software" ;;
    *) echo "Unknown package: no description available" ;;
```

```
esacesac
```

This script essentially acts as a package inspector that inspects whether or not a given software package is installed. It then proceeds to print relevant information about it, along with a small description.

The if-then-else block essentially inspects whether or not the package is installed. It uses a command that is similar to dpkg -s $PACKAGE (dpkg -l | grep), which essentially inspects the package database. If the command is successful (exit value 0), then it is assumed that the package is installed, and relevant information is displayed. If not, then the else part is executed, which states that the package is not installed.

The case statement is then used, which essentially branches depending on what $PACKAGE is. The patterns that we have defined here, such as httpd, mysql, vlc, etc., essentially match package names. We then print a small description of what that particular software is. If none of these match, then we have the default case.

```
┌──(aryan㉿kali)-[~]
└─$ ./package_inspector.sh
vlc is installed.
ii  vlc-bin                                3.0.23-1                                arm64        binaries from VLC
-------------------------------------
VLC: a powerful open-source media player for all formats

┌──(aryan㉿kali)-[~]
```

Script 3 — Disk and Permission Auditor

Concepts: for loop, arrays, du, ls -ld, awk, cut.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: R Harsha Reddy

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

echo "----------------------------------------"
echo "Software Config Directory Check"
echo "----------------------------------------"
```

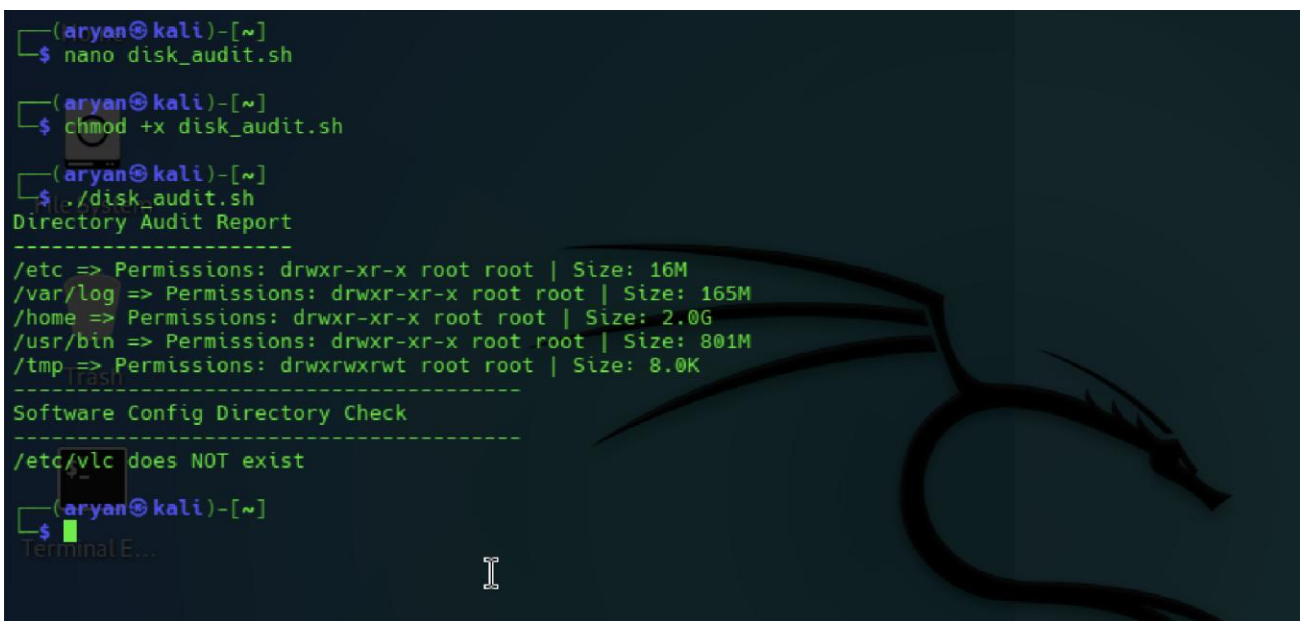
```
# Change this according to your chosen software
CONFIG_DIR="/etc/vlc"

if [ -d "$CONFIG_DIR" ]; then
    CONFIG_INFO=$(ls -ld $CONFIG_DIR | awk '{print $1, $3, $4}')
    echo "$CONFIG_DIR exists => Permissions: $CONFIG_INFO"
else
    echo "$CONFIG_DIR does NOT exist"
fi
```

This script checks and audits important system directories. The script does this by going through a set of directories defined in an array and then printing information about these directories.

the **`du -sh $DIR` command** is executed. This will display information about the total disk space used in the directory. The `-s` flag is used to display only total sizes, and `-h` is used to display sizes in human-readable format (KB, MB, etc.). The information is then passed through **`cut -f1`,** which means that only the first column will be displayed.

**`2>/dev/null` is used to suppress error messages. This is important to prevent error messages like “Permission denied” from appearing in case we try to execute this script in restricted directories.



Script 4 — Log File Analyzer

Concepts: while read loop, if-then, counter variables, command-line arguments ($1, $2).

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: R Harsha Reddy
# Usage: ./log_analyzer.sh /var/log/messages error

LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword is 'error'
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Do-while style retry if file is empty
while [ ! -s "$LOGFILE" ]; do
    echo "File is empty. Waiting for content..."
    sleep 2
done

# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "----------------------------------------"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo "----------------------------------------"

# Show last 5 matching lines
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

> This script reads a log file using the provided inputs, which are read at runtime, and processes the file line by line to find keyword matches.
>
> The variables **`$1` and `$2`** are **command-line arguments**. When you run the script with the following command:
>
> ```bash
>
> ./log_analyzer.sh sample.log error
>
> ```
>
> * `$1` becomes **`sample.log`** (log file)

* `$2` becomes **`error`** (keyword to search for)

The syntax for **`${2:-error}`** means: “use `$2` if provided, otherwise use `error`.” So, if you run the script with the following command

```bash

./log_analyzer.sh sample.log

```

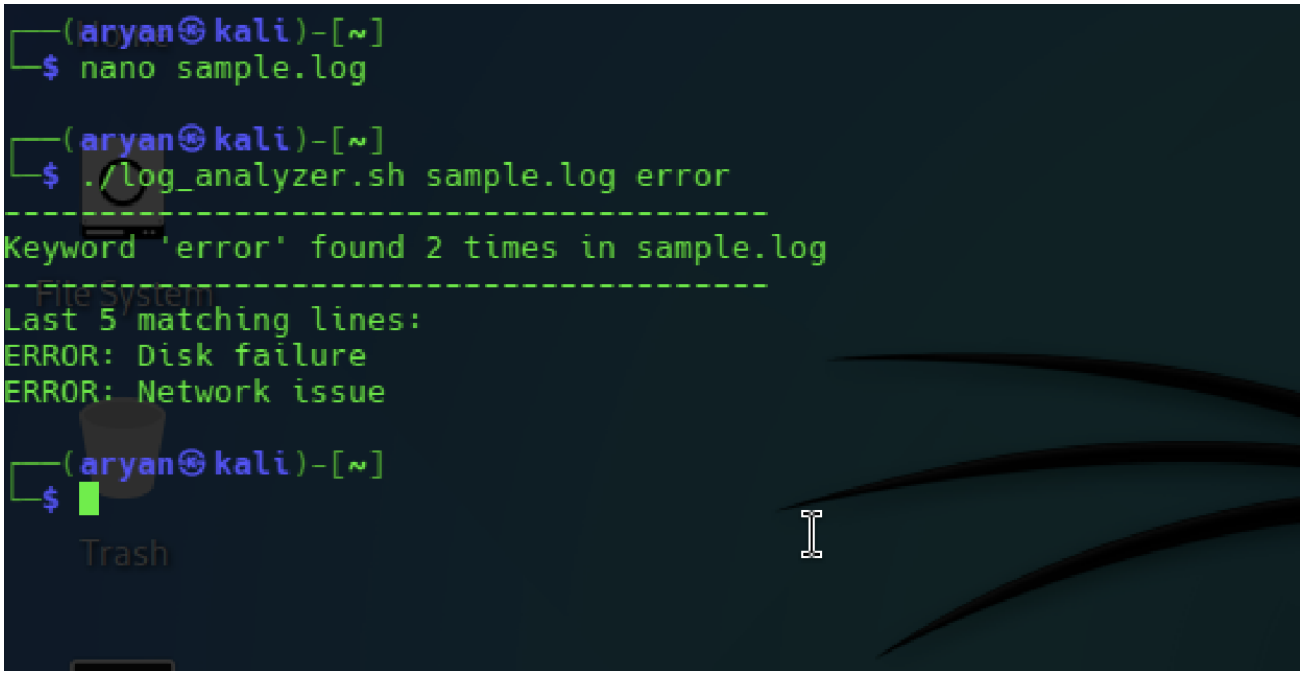
Inside the loop, the script searches for the keyword in the line and increments the count with **`COUNT=$((COUNT + 1))`**. This is called *arithmetic expansion* in Bash, which means you can perform arithmetic operations directly in the shell.

The loop also uses *input redirection* **`< "$LOGFILE"`** instead of using *cat*. This is more efficient because:

* It does not fork an extra process.

* It directly sends the file to the loop.

* It is generally recommended in shell scripting best practices.



Script 5 — The Open Source Manifesto Generator

Concepts: read for user input, string concatenation, writing to file with > and >>, date command, alias comment.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: R Harsha Reddy

# Alias example (optional use in terminal)
# alias manifesto="bash manifesto_generator.sh"

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Compose and write manifesto
echo "----------------------------------------" > "$OUTPUT"
echo "Open Source Manifesto — $DATE" >> "$OUTPUT"
echo "----------------------------------------" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "I believe in the power of open source. Using tools like $TOOL every day
reminds me that technology should be accessible to all." >> "$OUTPUT"
echo "To me, freedom means $FREEDOM — the ability to learn, modify, and share
without restrictions." >> "$OUTPUT"
echo "If given the opportunity, I would build $BUILD and share it with the world,
contributing to a community driven by collaboration and transparency." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "— $(whoami)" >> "$OUTPUT"

echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat "$OUTPUT"
```

> This script is designed to create a personal open source manifesto by prompting the user to input their responses, which are then written to a file in a formatted manner.
>
> The user input is taken using the **`read -p`** command. This command is used to ask the user to input something, and the `-p` option is used to display the prompt message on the same line, so the user knows what they are expected to input. This input is then assigned to variables like `TOOL`, `FREEDOM`, `BUILD`, etc.The text is formatted using the process of **string concatenation**, where the text is concatenated using the `echo`

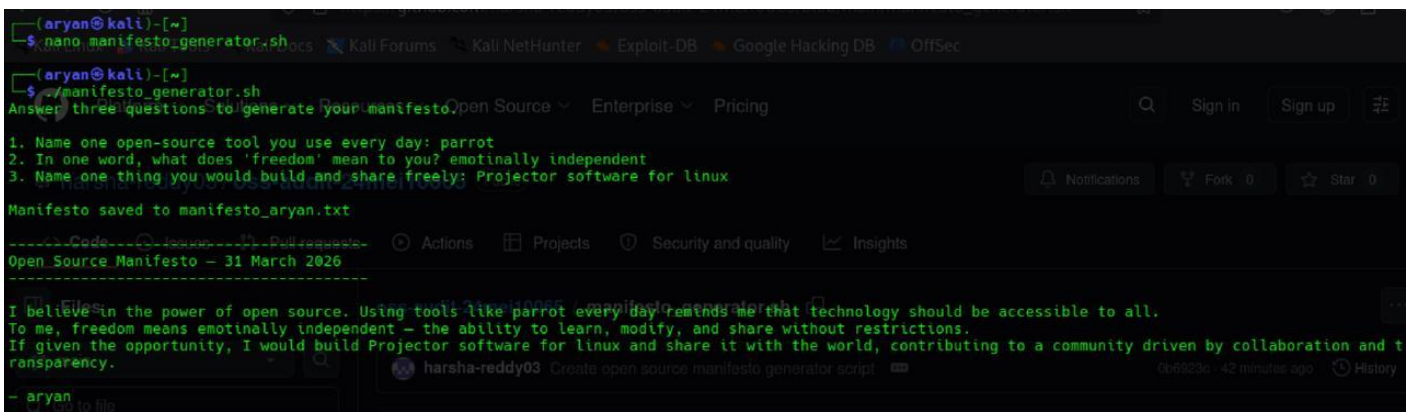
command. Each line is designed to add relevant sentences using the user's input, gradually building up the text to create a full paragraph, which is then written to the output file (for example, `manifesto_username.txt`).

The operators **`>` and `>>`** are used to write the text to the file:

* `>` is used to **create a new file or overwrite** the existing one (for the first line, where we want to start fresh).

* `>>` is used to **append** the text to the existing file (for the subsequent lines, where we do not want to overwrite the existing text).

The line that is commented out, which shows the **alias**, is where the user is shown how they could use the alias feature of the Bash shell. This feature is useful because the user could create their own alias using the alias command, which would enable the user to simply type `manifesto` at the terminal to execute the script.



References

1. GNU Project — The Free Software Definition. gnu.org/philosophy/free-sw.html

2. Open Source Initiative — The Open Source Definition. opensource.org/osd

3. Eric S. Raymond — The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar

4. VLC legal page: www.videolan.org/legal.html